

ШПАРГАЛКА

Linux: диагностика сервиса

Рабочий маршрут от симптома к доказанной причине: systemd, journal, process, socket, HTTP, permissions и filesystem.

Модуль	01 - Linux и systemd
Дата	23 сентября 2026
Охват	systemd + process + socket + HTTP + filesystem
Статус	Справочный материал для диагностики

Главная инженерная мысль

Одна команда должна отвечать на один вопрос. Сначала собери read-only evidence по всей цепочке, затем сделай минимальное изменение и повтори проверку из той же точки, где наблюдался симптом.

Слабый подход	Инженерный подход
restart до диагностики	сначала status и journal
порт открыт - значит всё хорошо	socket PID -> MainPID -> HTTP
chmod 777	service identity + точные права пути

1. Золотой маршрут

```

симптом
-> systemctl status
-> journal
-> process / identity
-> socket / PID
-> HTTP из нужной точки
-> config / permissions / filesystem
-> минимальное изменение
-> повторная проверка всей цепочки

```

Правило: **одна команда - один вопрос**. Не меняй систему, пока результат read-only проверки не различает хотя бы две гипотезы.

2. Первые пять вопросов

Вопрос	Формула команды	Ключевые поля	State change
Что видит systemd?	<code>systemctl status <unit></code>	Loaded, Active, Main PID	Нет
Почему процесс вышел?	<code>journalctl -u <unit> -n <N> --no-pager</code>	errno, path, PID, timestamp	Нет
Кто слушает порт?	<code>ss -lntp 'sport = :<port>'</code>	local address, PID, process	Нет
Отвечает ли HTTP?	<code>curl -i --max-time <sec> <url></code>	status, headers, body	Нет
Что реально загружено?	<code>systemctl cat <unit></code>	directives и drop-ins	Нет

Не начинай с restart: он меняет evidence и может временно скрыть причину.

3. systemd: состояние и конфигурация

Что различать

- active - процесс сейчас считается запущенным.
- enabled - unit включён в автозапуск.
- failed - последний запуск завершился ошибкой.
- inactive - процесс сейчас не работает; это не обязательно ошибка.
- MainPID - процесс, который systemd считает главным.

Полезные формулы

Формула	Вопрос	Типичная ошибка
<code>systemctl status <unit> --no-pager -l</code>	Каков текущий lifecycle?	Считать active доказательством HTTP
<code>systemctl show <unit> -p MainPID -p User -p Group</code>	Кто и от чьего имени работает?	Смотреть только имя процесса
<code>systemctl cat <unit></code>	Какой unit и drop-ins загружены?	Читать только основной файл
<code>systemctl list-units --type=service --all</code>	Есть ли похожий unit?	Путать unit file и живой process
<code>systemctl status <PID></code>	Какой unit владеет PID?	Убивать PID до идентификации

Когда нужен daemon-reload

daemon-reload нужен после изменения unit-файла или drop-in. Изменение файла, который приложение читает при старте (EnvironmentFile), обычно требует restart процесса, но не перечитывания unit-структуры.

```
unit/drop-in изменён -> daemon-reload -> restart
env/app config изменён -> restart
только диагностика -> ничего не перезапускать
```

4. Journal и ошибки

Читай ошибку снизу вверх до первой конкретики: errno, путь, адрес, порт или имя dependency. Traceback важен не целиком, а в месте первого системного отказа.

Сообщение	Обычно означает	Следующая read-only проверка
Permission denied	Нет нужного r, w или x	identity + namei -l + test
Address already in use	Другой socket уже владеет адресом	ss по точному порту
Connection refused	На destination никто не слушает	listener и target URL
Connection timed out	Пакет/ответ теряется по пути	route, firewall, remote listener
No space left on device	Закончились blocks или inodes	df -h и df -i
status=2 / invalid argument	Неверный аргумент или config	полная строка ошибки + config source

5. Process, socket и HTTP

Три уровня нельзя смешивать:

Уровень	Что он доказывает	Чем проверить
Process	Код запущен	systemd MainPID, ps
Socket	Процесс принимает соединения на address:port	ss -lntp
HTTP	Приложение выполнило протокол и контракт	curl -i

Bind-address определяет, через какие интерфейсы доступен listener:

- 127.0.0.1 - только внутри этой машины;
- конкретный IP - только через соответствующий интерфейс;
- 0.0.0.0 - все IPv4-интерфейсы, но firewall всё ещё может блокировать путь.

curl доказывает доступность только из точки, где запущен. Локальный успешный запрос не гарантирует доступ с Мас, балансировщика или другой VM.

6. Права и путь

Числа chmod

```
r = 4, w = 2, x = 1
owner | group | others
750 = rwx r-x ---
640 = rw- r-- ---
770 = rwx rwx ---
```

Для каталога:

- `r` - увидеть список имён;
- `w` - создавать и удалять записи;
- `x` - пройти через каталог и обратиться к объекту внутри.

Для создания файла нужны `w+x` на каталоге назначения. Проверяй весь путь:

Формула	Вопрос	State change
<code>namei -l <path></code>	Где на пути потеряны права?	Нет
<code>ls -ld <dir></code>	Кому принадлежит сам каталог?	Нет
<code>ls -l <file></code>	Кому принадлежит файл?	Нет
<code>sudo -u <user> test -w <dir>; echo \$?</code>	Может ли service user писать?	Нет
<code>id <user></code>	В какие группы входит пользователь?	Нет
<code>chown <owner>:<group> <path></code>	Сменить владельца или группу	Да
<code>chmod <mode> <path></code>	Изменить mode bits	Да

`chown` и `chmod` не взаимозаменяемы. Избегай `777`: исправляй точный объект и минимальный набор битов.

7. Filesystem: blocks и inodes

Файловая система имеет как минимум два ограниченных ресурса:

Ресурс	Что расходует	Проверка
Blocks	Содержимое файлов	<code>df -h <path></code>
Inodes	Количество файлов и каталогов	<code>df -i <path></code>

Ты можешь иметь свободные гигабайты и не суметь создать пустой файл, если `IFree=0`. При очистке сначала формируй точную выборку и печатай её:

```
find <root> -xdev -type f -name '<mask>' <age-filter> -print
```

Только после проверки списка добавляется действие удаления. Не применяй широкую маску и рекурсивное удаление к неизвестному каталогу.

8. Dependencies systemd

`After=` задаёт порядок запуска только для units, уже включённых в транзакцию. Он не означает «запусти dependency».

`Requires=` создаёт обязательную связь: при старте основного unit systemd также добавляет зависимость. Часто нужны обе директивы:

```
[Unit]
Requires=<dependency>.service
After=<dependency>.service
```

После создания drop-in: проверить `systemctl cat`, выполнить `daemon-reload`, перезапустить и доказать состояние обеих служб.

9. Минимальное исправление

Хороший fix:

- меняет только доказанную причину;
- сохраняет принцип минимальных прав;
- управляет процессом через его supervisor;
- не уничтожает исходные evidence;
- имеет понятный rollback;
- проверяется несколькими независимыми сигналами.

Плохие универсальные реакции: `chmod 777`, `kill` неизвестного PID, `restart` всех служб, отключение `firewall`, рекурсивное удаление и редактирование кода до чтения журнала.

10. Проверка после исправления

Минимальная лестница доказательств:

1. `unit` находится в ожидаемом состоянии;
2. `MainPID` существует и соответствует нужному процессу;
3. `listener` принадлежит этому `MainPID`;
4. `HTTP` возвращает нужный `status code`;
5. `payload` соответствует контракту;
6. `journal` текущего PID не содержит новой ошибки;
7. отдельный `checker` проходит, если он предусмотрен.

Проверяй с той же точки, где был симптом. Для внешней недоступности нужен внешний клиент; локальный `curl` недостаточен.

11. Быстрая карта решений

Симптом	Не спешить делать	Сначала доказать
Unit failed	редактировать первый попавшийся config	точную ошибку и source config
Active, но curl не работает	перезапустить	address, port, точку клиента
Permission denied	ставить 777	service identity и проблемный компонент пути
Port busy	делать <code>kill -9</code>	PID → unit → назначение процесса
Errno 28	удалять большие файлы	blocks или inodes закончились
Dependency 503	менять URL	dependency active, socket, HTTP, graph

12. Одностраничный чек-лист

- [] Воспроизвёл симптом и записал точную команду/время.
- [] Посмотрел `systemctl status` и `MainPID`.
- [] Прочитал `journal` текущего запуска.
- [] Нашёл `User/Group`, `ExecStart`, `EnvironmentFile`.
- [] Проверил `listener` и его `PID`.
- [] Сопоставил `listener PID` с `MainPID/unit`.
- [] Проверил `HTTP status` и `payload` из нужной точки.
- [] При `filesystem`-ошибке проверил `path permissions`, `df -h` и `df -i`.
- [] Сформулировал одну доказанную `root cause`.
- [] Сделал минимальный `fix` с понятным `rollback`.
- [] Повторил всю цепочку и сохранил `evidence`.